

Artificial Intelligence for robotics

Part 2: the neuron

Simon Gay

Artificial Intelligence for Robotics: the neuron

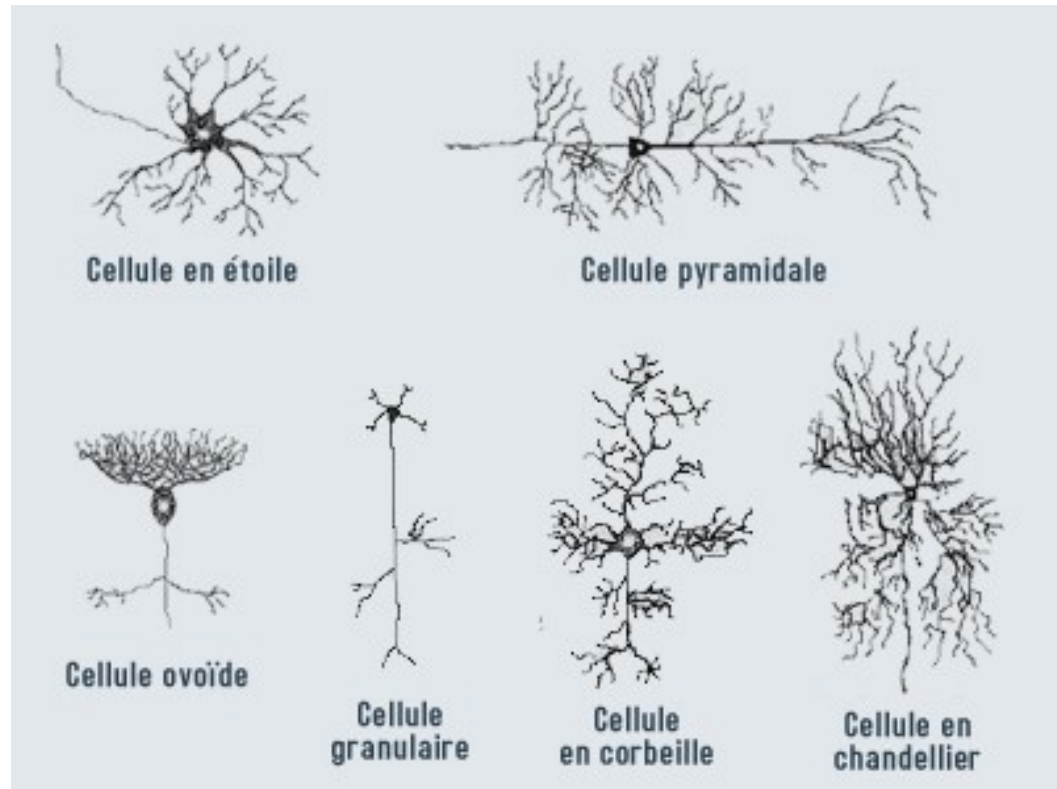
- Artificial intelligence tries to implement several properties of human or animal intelligence
- The only known form of intelligence emerges from the most incredible, but also the most mysterious organ of living beings: the brain



- Case of human brain:
 - ~80-100 billion neurons
 - *As much glial cells (many of which are astrocytes)*
 - $\sim 1,5 \times 10^{14}$ connections
 - 10.000 to 50.000 neurons and 100 to 500 millions connections per mm^3
 - Many specialized areas
- Even animals cannot be fully mapped and simulated (especially due to the complexity of neurons and their connections)
- Let's analyze the fundamental component of the brain : the neuron

Artificial Intelligence for Robotics: the neuron

Main neuron types (>200 types) :

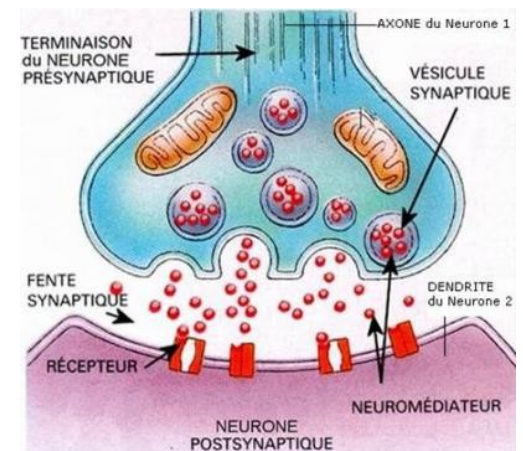
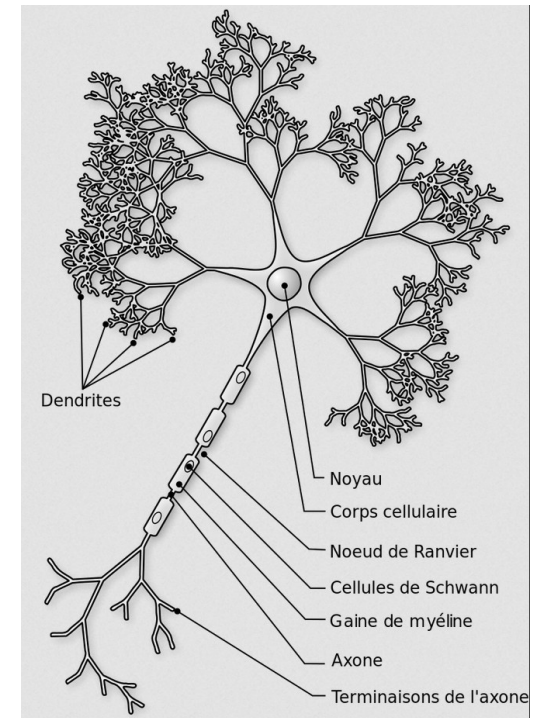


New types are regularly discovered:

- Rosehip in 2018
- Campana in 2022

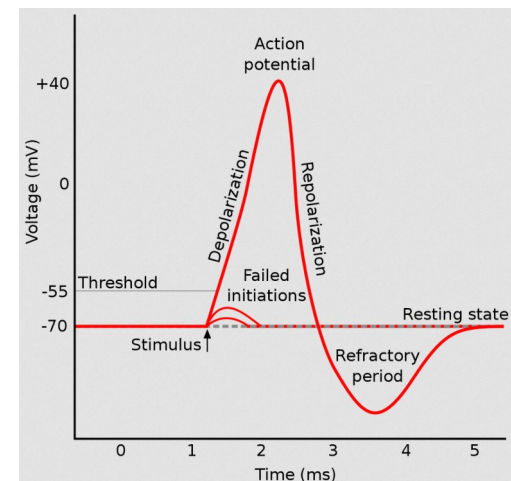
Artificial Intelligence for Robotics: the neuron

- Anatomy of a unipolar neuron:
 - A body (also called soma)
 - Inputs: the *dendrites* (around 7000 per neuron)
 - Output: a unique *axon*
- Neurons are connected through *synapses*
 - A synapse links the axon of a neuron and the dendrite of another neuron
 - Signal transmission is unidirectional
 - But sometime bi-directional in some types
 - No direct contact between neurons
 - Emitter side:
 - the neuron produces neurotransmitters
 - Receiver side:
 - the neuron detects the neurotransmitters
 - The more receivers there are, the stronger the signal received
 - Each synapse can contains multiple types of neurotransmitters.



Artificial Intelligence for Robotics: the neuron

- A neuron generates pulses of 1 to 2 milliseconds
 - Its activity can be defined as the frequency of pulse emission.
- A neuron has a potential of -70mV relative to ambient environment (rest tension) due to the presence of ions (Na^+ , K^+ , Cl^- et Ca^{2+})
- When receiving a synaptic signal, its tension increases or decreases (depending of neurotransmitter type)
- When the tension becomes greater than the threshold of -55 mV :
 - the membrane channels open to allow ions to pass through (depolarization)
 - A signal (action potential) of +100 mV is transmitted through the axon
 - The synaptic vesicles release neurotransmitters
 - The neuron returns to its resting potential (-70 mV)
- The signal (and thus information) is thus related to the synaptic connection strength between neurons of the network



Artificial Intelligence for Robotics: the neuron

Neuron learning mechanism

- Most common mechanism among neurons: principle of Hebb (Donald Hebb , 1949)
 - Neurons firing simultaneously reinforce their connection.
 - → When two connected neurons depolarize simultaneously, the synapse increases the number of neurotransmitter vesicles.
 - Consequence: when the emitter neuron's signal has a great incidence on the activation of receiver neuron, the connection must be reinforced.
- Another mechanism: the long-term polarization, mostly used by neuron of the hippocamp (area responsible of long term memory and navigation)
 - In case of high frequency stimulation, the synaptic connection is reinforced for a long period of time

Artificial Intelligence for robotics

Part 3: the formal neuron

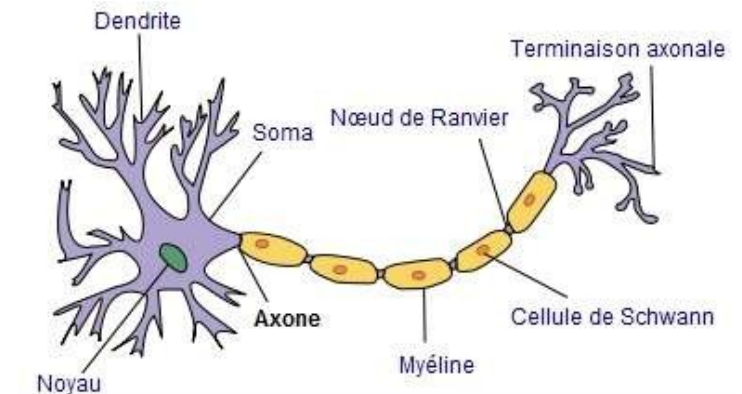
Simon Gay

Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**

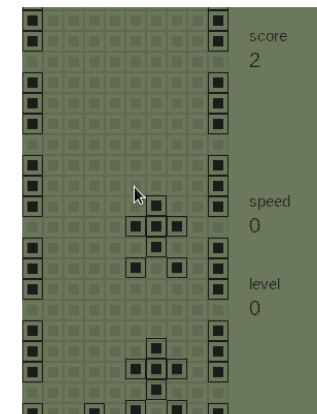
- biological neuron:

- Dendrites collecting input signal
 - A body integrating input signals
 - An axon transmitting the signal
 - Activity defined by impulsion (frequency)



- The model must be simplified to work on a computer

- /\ The formal neuron is an VERY simplified version of the biological neuron



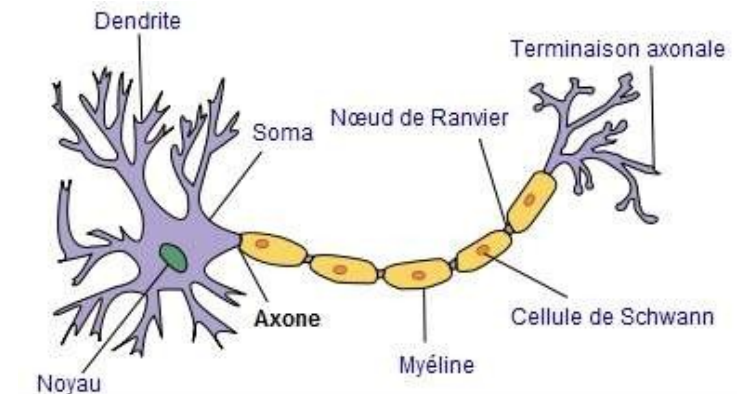
<https://github.com/tshiamobhuda/Racing>

Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**

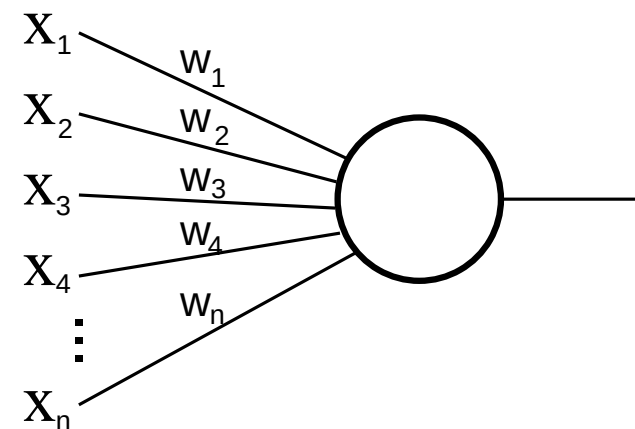
- biological neuron:

- Dendrites collecting input signal
 - A body integrating input signals
 - An axon transmitting the signal
 - Activity defined by impulsion (frequency)



- The model must be simplified to work on a computer

- The frequency is replaced by a real value
 - Input: a vector of values
 - Vector $X = \{x_1, x_2, x_3, \dots, x_n\}$
 - A vector of reals for synapses
 - Vector $W = \{w_1, w_2, w_3, \dots, w_n\}$

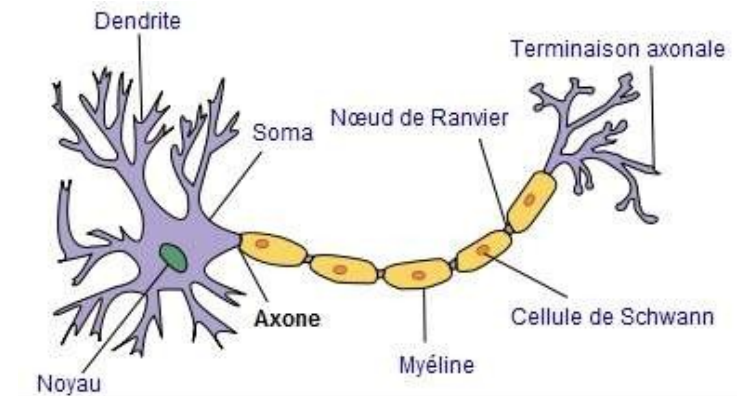


Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**

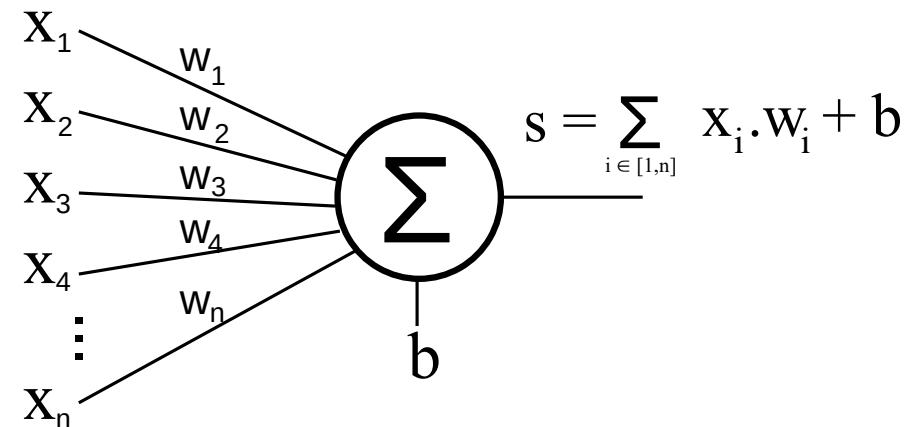
- biological neuron:

- Dendrites collecting input signal
 - A body integrating input signals
 - An axon transmitting the signal
 - Activity defined by impulsion (frequency)



- The model must be simplified to work on a computer

- For integration of input values:
 - Sum of inputs values weighted by synaptic weights
 - A bias b

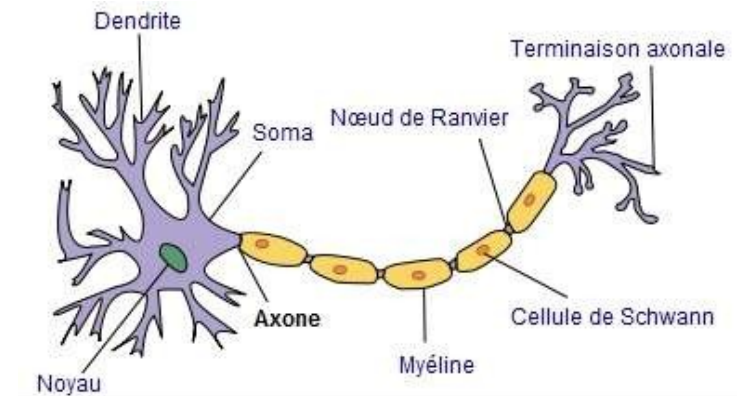


Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**

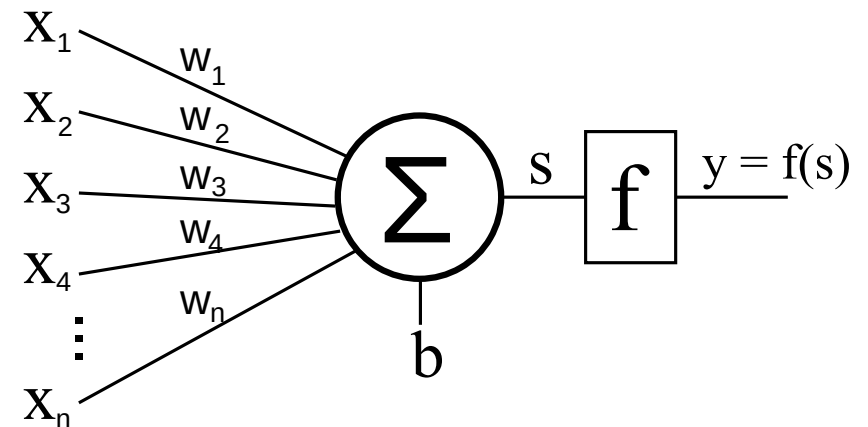
- biological neuron:

- Dendrites collecting input signal
 - A body integrating input signals
 - An axon transmitting the signal
 - Activity defined by impulsion (frequency)



- The model must be simplified to work on a computer

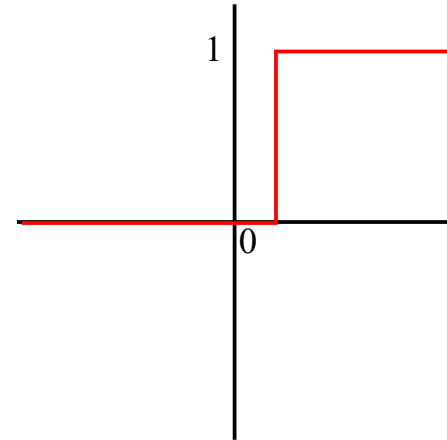
- The output passes through an activation function
 - Allows bounding the output value



Artificial Intelligence for Robotics: the formal neuron

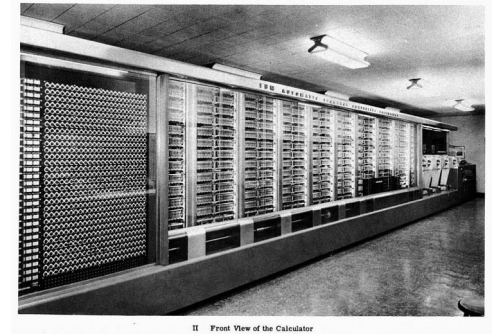
- Which activation function ?

- Threshold function: $S = 1$ if $y > \text{threshold}$, $S=0$ else



(note: the threshold value is not important because of the bias)

- Function used by first formal neuron models (Perceptrons, proposed by Warren McCulloch and Walter Pitts in **1943**)
 - Some limitations:
 - We cannot compare output values of multiple neurons
 - The strength of the signal cannot be transmitted to other neurons.

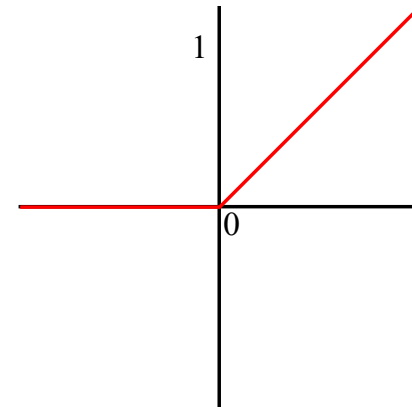
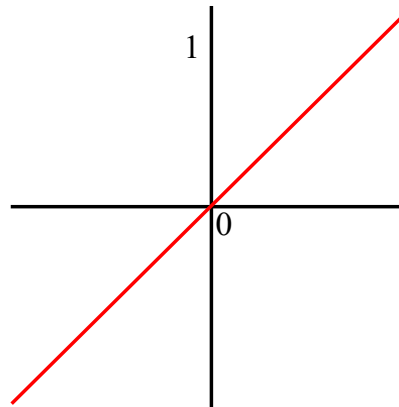


Artificial Intelligence for Robotics: the formal neuron

- Which activation function ?

- Linear function and ReLU (Rectified Linear Unit)

- $y = x$
- $y = \max(0, x)$



- Very simple
- ReLU remove negative values
 - Widely used in deep learning models
- The output can be greater than 1
- Gradient is constant

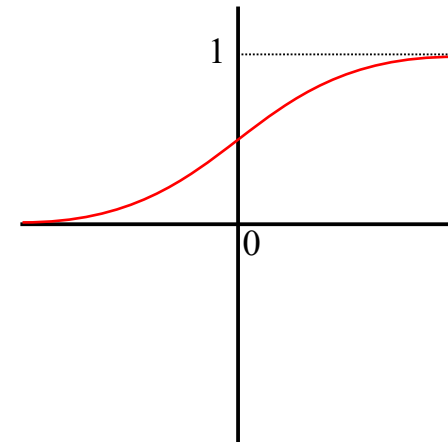
Artificial Intelligence for Robotics: the formal neuron

- Which activation function ?

- Sigmoid function

- $y = \frac{1}{1 + e^{-x}}$

(note: tanh is also used)



- More complex to implement and compute
- Great values generate a result close to 1 (or 0 if negative)
- gradient tends to 0 for great values
 - Stabilization of weights (variations becomes increasingly small, but still present)
 - But gradient “loss” (a problem when using a large number of layers)
- Another great advantage we will see later

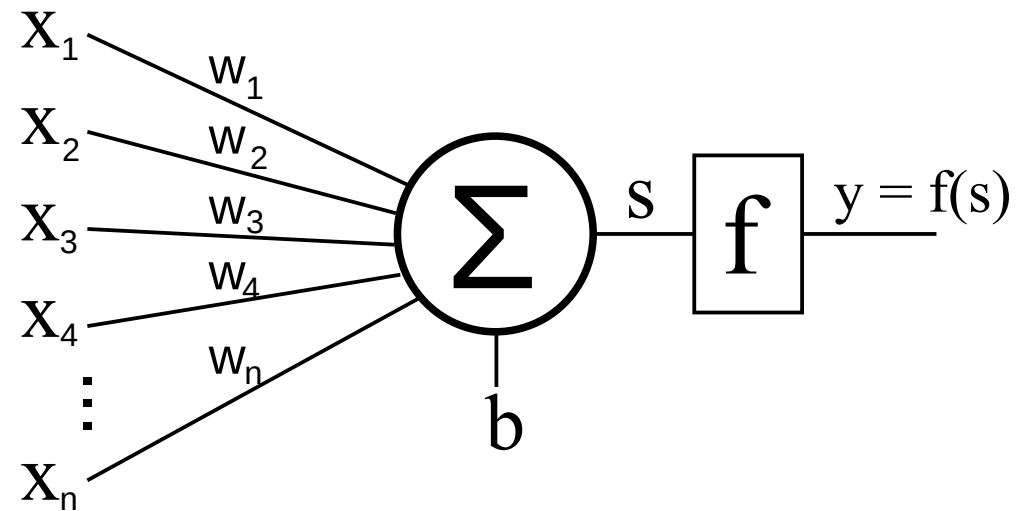
Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**

- The formal neuron
 - An input vector X
 - A weight vector W
 - An additional bias b
 - Equivalent to an input at 1

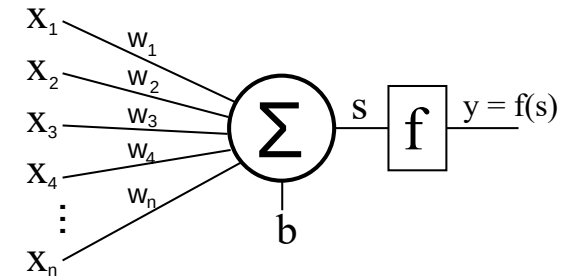
$$s = \sum_{i \in [1,n]} X_i \cdot W_i$$

- An activation function
 $Y = f(S)$



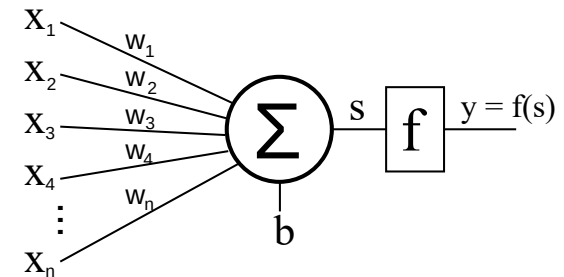
Artificial Intelligence for Robotics: the formal neuron

- **Supervised learning methods**



- Hebb rule (1949) : “*cells that fire together, wire together*”
 - $w_i^{t+1} = w_i^t + \alpha \cdot x_i \cdot r$: if r and x_i have the same sign, w_i is reinforced
 α is the learning rate (limits weight variations)
- Delta rule: changes each weight w_i proportionally to:
 - The input value (the higher the value of the input, the greater its impact on the result)
 - The difference between result and expected output (the greater the difference, the more the weights need to be adjusted)

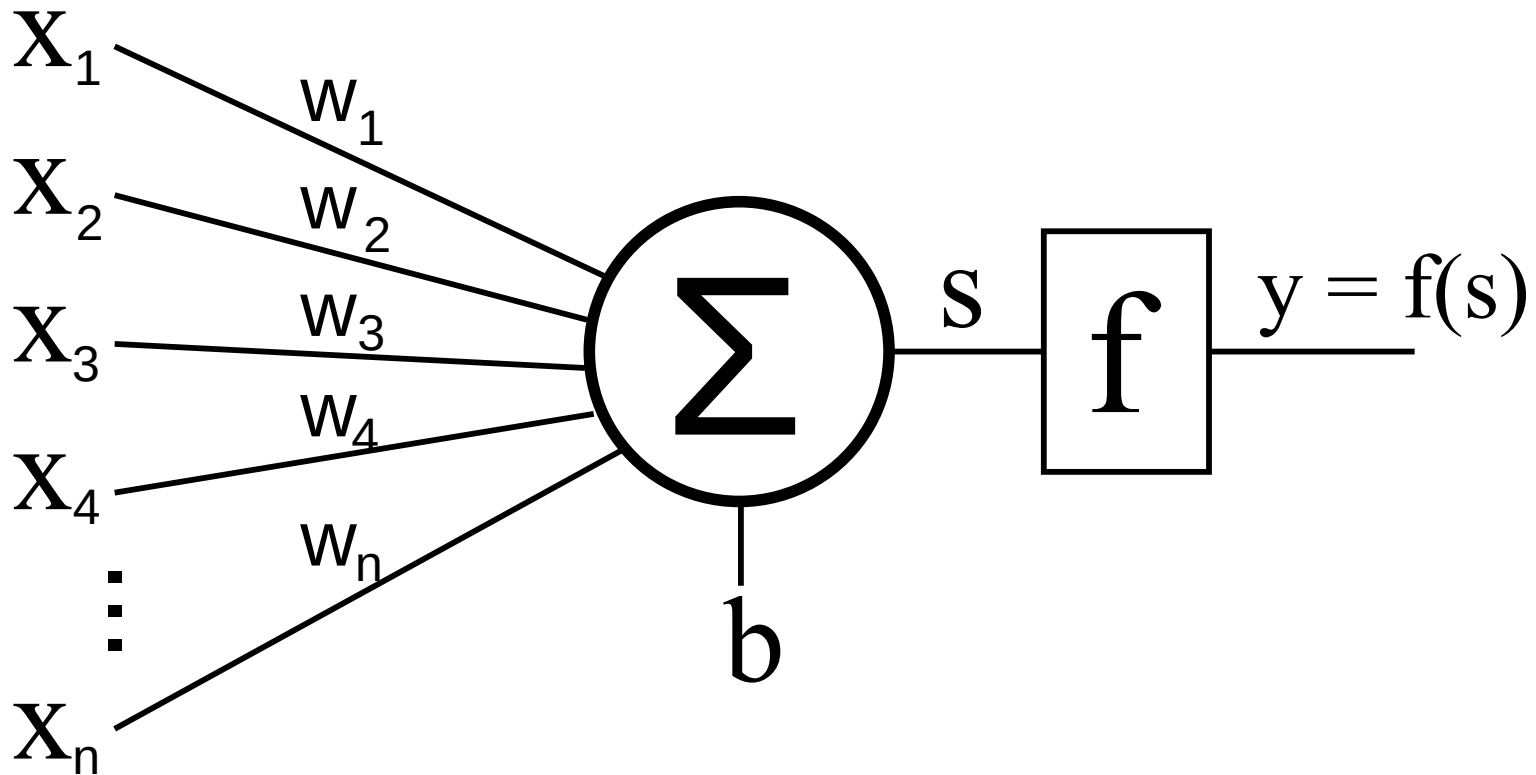
Artificial Intelligence for Robotics: the formal neuron



- **Supervised learning methods**

- Delta rule: we compute the difference between expected result r and output y of the neuron:
 - $\Delta = r - y$
 - Perceptron (Rosenblatt, 1957): Treshold function, thus $\Delta \in \{ -1 ; 0 ; 1 \}$
 - Widrow-Hoff rule (1960): continuous activation function, thus $\Delta \in \mathbb{R}$
- Each weight is updated:
 - Principles: weights are adjusted proportionally to:
 - Input value
 - Différence (Δ) between output and result
 - $w_i^{t+1} = w_i^t + \alpha \cdot x_i \cdot \Delta$
 α is the learning rate. Same update for the bias b
- The learning process is performed on a (very) high number of examples

Artificial Intelligence for Robotics: the formal neuron

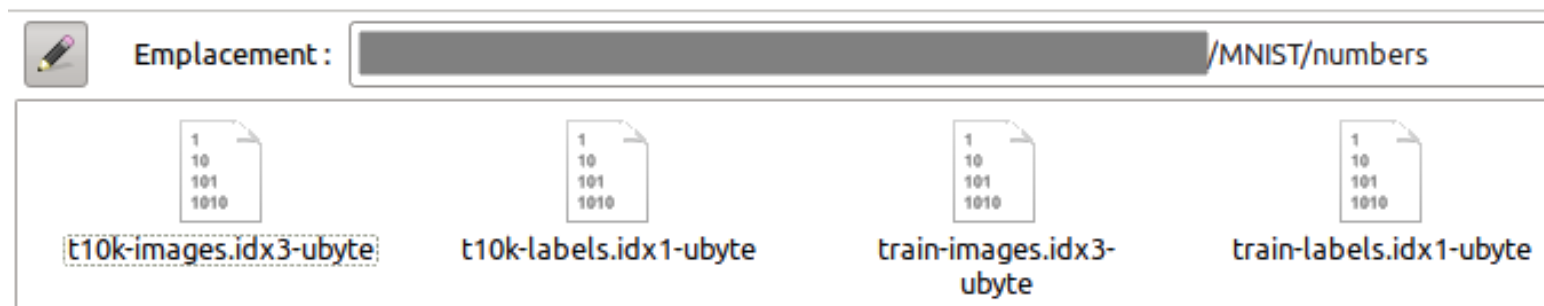


Let's code !

Artificial Intelligence for Robotics: the formal neuron

- **Environment**

- Create a new folder 'workspace_AI' as our Java workspace
- Download on https://github.com/gaysimon/AI_for_robotics the project archive '**1_Neuron.zip**'
- Download the four dataset files on <https://github.com/fgnt/mnist> ('.gz' files), **unzip** them and place them in a folder and subfolder "**MNIST/numbers/**" on your personal space

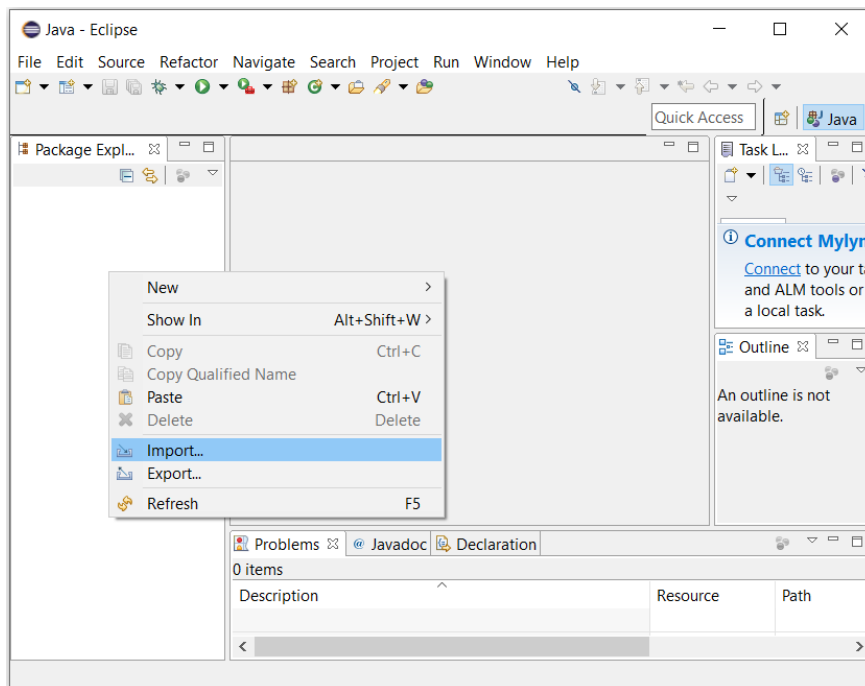


- Open Eclipse and use your workspace folder
 - In the welcome panel, click on '**workbench**' arrow

Artificial Intelligence for Robotics: the formal neuron

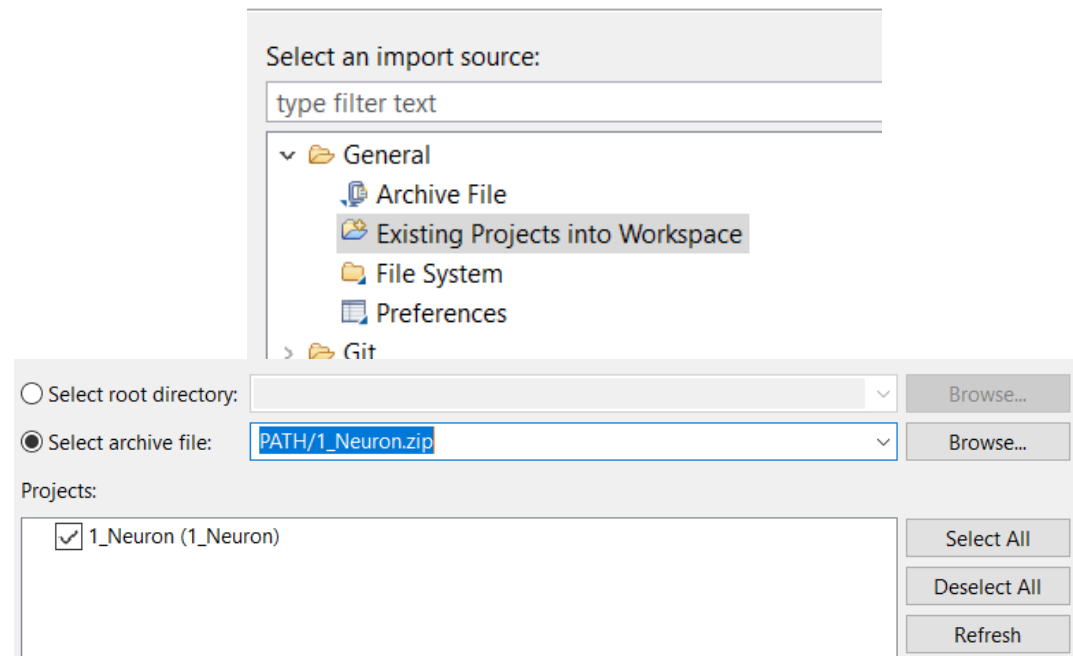
- **Environment**

- In the Package Explorer, right-click and select “Import”
- Select “General” then “Existing Projects into Workspace”, then click “Next”
- Check “Select archive file”, select the archive “1_Neuron.zip” on your personal space, check the project “1_Neuron” and then click on “Finish”



Select

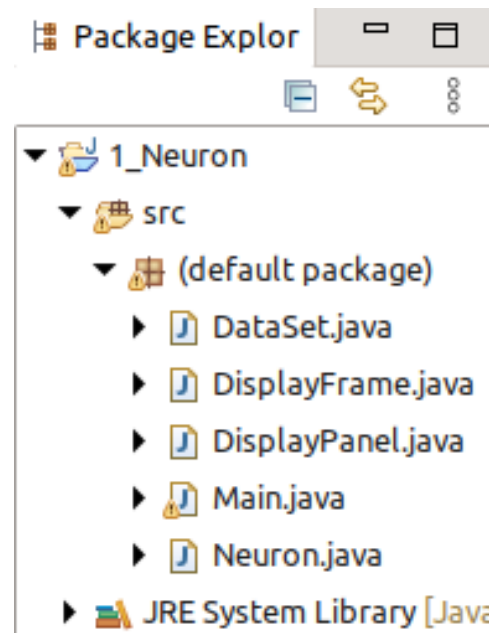
Create new projects from an archive file or directory.



Artificial Intelligence for Robotics: the formal neuron

- **Environment**

- The project contains 5 classes:
 - **Main**: contains the main function, perform the training and evaluation process
 - **Neuron**: functions implementing a formal Neuron
 - **DataSet**: contains a function to load datasets
 - **DisplayFrame** and **DisplayPanel**: display panel



Artificial Intelligence for Robotics: the formal neuron

- **Environment**

- In the **DataSet** class:

- Define the path to the MNIST/numbers/ folder (do not forget the "/" at the end)
 - /\ on Windows, use "\\" as folder separators

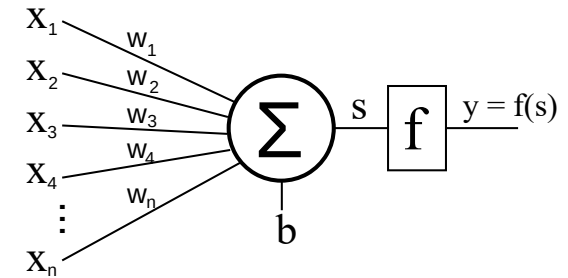
```
6 public class DataSet {  
7  
8     public static String PATH="Path_to_dataset/MNIST/numbers/"; // path to images  
9 }
```

- Start the project with button . If everything is right, you should see the display panel



Artificial Intelligence for Robotics: the formal neuron

- **Implementing a formal neuron**



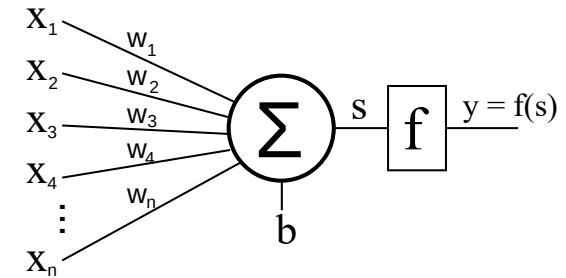
- Lets add the data structures :
 - A vector of float for weights
 - A float for the bias
 - A float to store the output
 - A float for the delta (→ used for evaluation)
- Then, we define the constructor:
 - The only parameter is the number of weights
 - The weight vector can then be initialized

```
1
2 public class Neuron {
3
4     public float[] weights;
5     public float bias;
6     public float output=0;
7     public float delta=0;
8
9     public Neuron() {
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
public Neuron(int size){
    weights=new float[size];
}
```

Artificial Intelligence for Robotics: the formal neuron

- Implementing a formal neuron



- The output function **compute**

$$s = \sum_{i \in [1,n]} x_i \cdot w_i + b$$

- Weighted sum of inputs
- Uses the input vector X as parameter
- Return the result (float)

- Then we add a sigmoid activation function:

$$y = \frac{1}{1 + e^{-x}}$$

- Update the compute function !

```
public float compute(float[] img) {  
  
    float sum=0; // weighted sum  
    for (int i=0; i<img.length; i++) {  
        sum+= img[i] * weights[i] ;  
    }  
    sum+=bias; // don't forget the bias  
    output=activation(sum);  
    return output;  
}
```

```
public float activation(float x) {  
    return (float) ( 1 / (1+Math.exp(-x)));  
}
```

Artificial Intelligence for Robotics: the formal neuron

- **Implementing a formal neuron**

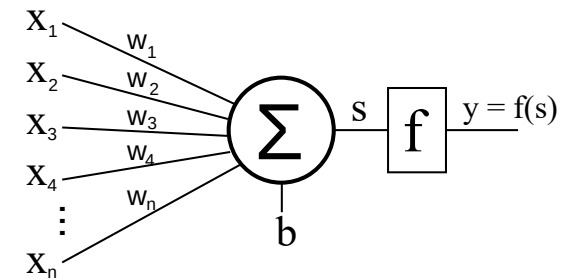
- The learning function **learn**

- Delta rule:

- $\Delta = r - y$

- $w_i^{t+1} \leftarrow w_i^t + \alpha \cdot x_i \cdot \Delta$

- We add a learning rate set at 0.001
 - Function needs input vector AND expected result
 - Record the delta with the variable

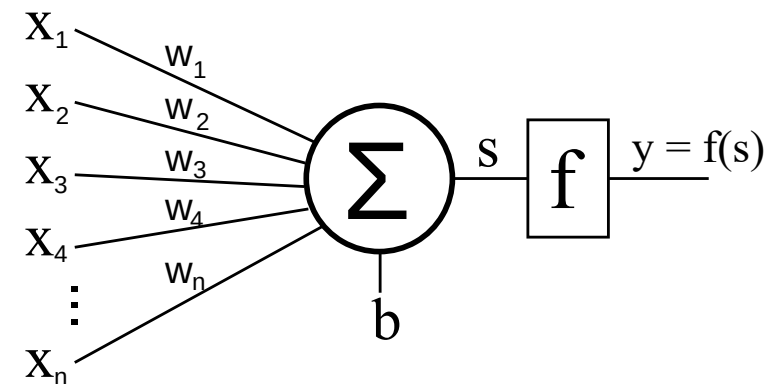


```
2 public class Neuron {
3
4     public float learnRate=0.001f;
5
6     public float[] weights;
7
8     public void learn(float[] img, int res){
9
10        delta=res-output; // compute error
11
12        for (int i=0;i<weights.length;i++){
13            weights[i]+= learnRate * img[i] * delta;
14        }
15        bias+= learnRate * delta; // update bias
16    }
17 }
```

- **The neuron is complete !**

Artificial Intelligence for Robotics: the formal neuron

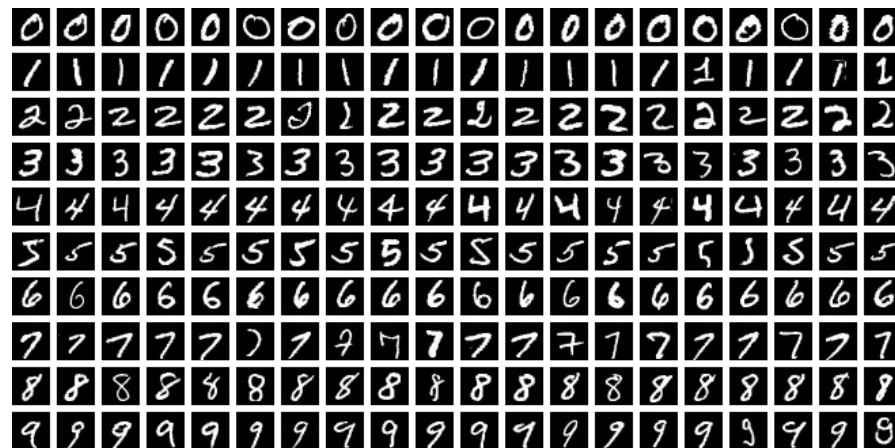
```
1
2 public class Neuron {
3
4     public float learnRate=0.001f;
5
6     public float[] weights;
7     public float bias;
8     public float output=0;
9     public float delta=0;
10
11 public Neuron(int size){
12     weights=new float[size];
13 }
14
15 public float compute(float[] img){
16     float sum=0; // weighted sum
17     for (int i=0; i<img.length; i++){
18         sum+= img[i] * weights[i] ;
19     }
20     sum+=bias; // don't forget the bias
21     output=activation(sum);
22     return output;
23 }
24
25 public void learn(float[] img, int res){
26     delta=res-output; // compute error
27     for (int i=0; i<weights.length; i++){
28         weights[i]+= learnRate * img[i] * delta;
29     }
30     bias+= learnRate * delta; // update bias
31 }
32
33 // activation function Sigmoid
34 public float activation(float x){
35     return (float) ( 1 / (1+Math.exp(-x)));
36 }
37 }
```



Artificial Intelligence for Robotics: the formal neuron

- **Training process**

- The training process is performed by the Main class.
- We will use a famous dataset: MNIST
 - *Modified National Institute of Standards and Technology*
 - Created in 1994, widely used since 1998 for benchmarks
 - Contains 60 000 hand-written labeled digits for training and 10 000 for tests
 - Images of 28x28 pixels
 - The dataset files encode images as a unique sequence of *int* values (0-255)



Artificial Intelligence for Robotics: the formal neuron

- **Training process**

- The function **DataSet.setData()** (provided) read a dataset and its labels and store them in matrices:
 - `float[][] matrixImages`: each line is a vector of 28x28 float (0-1) values encoding one image
 - `int[] matrixLabels`: each value is a label indicating the digit in each image (0 to 9)
- The training process works as follow:
 - Load training dataset
 - For each dataset input:
 - Define the expected output of the neuron (0 or 1)
 - Compute the neuron's result
 - Update weights

Artificial Intelligence for Robotics: the formal neuron

- Training process

- First, set the size of the neuron
 - Vector of 28x28 values

```
33 ///////////////  
34 public Main() {  
35  
36     // initialize structures  
37     neuron=new Neuron(size_x*size_y);  
38 }
```

- Then, we will create the training loop:
 - We use the global variable test (linked to the display panel)

```
// initialize display frame  
display=new DisplayFrame(this);  
  
for (test=0;test<matrixImages.length;test++){  
  
    // TODO : learning  
  
    //try {Thread.sleep(10);  
    //} catch (InterruptedException e) {e.printStackTrace();}  
}  
  
System.out.println("learning completed");
```

Artificial Intelligence for Robotics: the formal neuron

- Training process

- A neuron has an output between 0 and 1: our neuron can thus recognize only one digit. In global variables, select a digit to be recognized (0 to 9)

```
public static int number=3;           // selected digit for test
```

- The expected output can be defined: if the label is equal to *number*, the output should be 1, otherwise, it should be 0.
- Then, we can use function compute to get the output and learn to train the neuron

```
for (test=0;test<matrixImages.length;test++){  
  
    // define expected result  
    int res=0;  
    if (matrixLabels[test]==number) res=1;  
  
    // process neuron  
    neuron.compute(matrixImages[test]);  
    neuron.learn(matrixImages[test], res);  
  
    display.repaint(); // update display panel  
}
```

Artificial Intelligence for Robotics: the formal neuron

- Training process

- You can uncomment the neuron weights display in the DisplayPanel class

```
for (int i=0;i<Main.size_x;i++){  
    for (int j=0;j<Main.size_y;j++){  
        val=(int) (main.neuron.weights[i+Main.size_x*j]*50)+128;  
        if (val<0) val=0;  
        if (val>255) val=255;  
        g.setColor(new Color(val,val,val));  
        g.fillRect(180+5*i, 10+5*j, 5, 5);  
    }  
}
```

- You can also uncomment the delay in the loop to observe digit images

```
display.repaint(); // update display panel  
  
try {Thread.sleep(10);  
} catch (InterruptedException e) {e.printStackTrace();}  
}
```



Artificial Intelligence for Robotics: the formal neuron

- Training process

- Processing the dataset once is often not sufficient. We will process it multiple times. Each dataset training is called an **epoch**
- We add a loop to perform 20 epochs

```
// initialize display frame
display=new DisplayFrame(this);

// epoch loop
for (epoch=0;epoch<20;epoch++){

    // dataset loop
    for (test=0;test<matrixImages.length;test++){

        // define expected result
        int res=0;
        if (matrixLabels[test]==number) res=1;

        // process neuron
        neuron.compute(matrixImages[test]);
        neuron.learn(matrixImages[test], res);

        display.repaint(); // update display panel

        //try {Thread.sleep(10);
        //} catch (InterruptedException e) {e.printStackTrace();}

    }

}
```

Artificial Intelligence for Robotics: the formal neuron

- Training process

- We then measure performances by collecting all delta values of an epoch

```
// epoch loop
for (epoch=0;epoch<20;epoch++){

→ float sumdelta=0; // initialize the sum of errors

    // dataset loop
    for (test=0;test<matrixImages.length;test++){
        // define expected result
        int res=0;
        if (matrixLabels[test]==number) res=1;
        // process neuron
        neuron.compute(matrixImages[test]);
        neuron.learn(matrixImages[test], res);

→ sumdelta+=Math.abs(neuron.delta); // add error (absolute value)

        display.repaint(); // update display panel
        //try {Thread.sleep(10);
        //} catch (InterruptedException e) {e.printStackTrace();}
    }

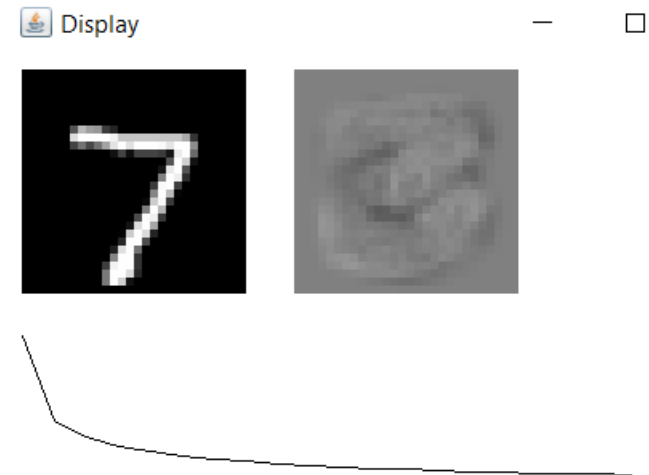
    // display sum of errors
→ System.out.println("epoch n°"+epoch+" : "+(sumdelta/matrixImages.length));
}
```

Artificial Intelligence for Robotics: the formal neuron

- Training process

- We then measure performances by collecting all delta values of an epoch

```
epoch n°0 : 0.08726755
epoch n°1 : 0.060714222
epoch n°2 : 0.055588827
epoch n°3 : 0.05288086
epoch n°4 : 0.051110104
epoch n°5 : 0.04982587
epoch n°6 : 0.04883459
epoch n°7 : 0.048038695
epoch n°8 : 0.047379997
epoch n°9 : 0.04682368
epoch n°10 : 0.046345204
epoch n°11 : 0.04592855
epoch n°12 : 0.04556147
epoch n°13 : 0.04523546
epoch n°14 : 0.044943605
epoch n°15 : 0.04468028
epoch n°16 : 0.044441562
epoch n°17 : 0.044223938
epoch n°18 : 0.044024635
epoch n°19 : 0.043841813
```



Artificial Intelligence for Robotics: the formal neuron

- **Evaluation**

- The evaluation must be performed on different inputs than for training
- MNIST proposes a second dataset of 10 000 images
- After the training loop, add another dataset loop (no epoch), with variable *test*
- In this loop, we only compute the neuron output and display the result

```
// load evaluation dataset
setData("t10k-images.idx3-ubyte", "t10k-labels.idx1-ubyte");

// TODO : evaluation
for (test=0; test<matrixImages.length; test++) {

    System.out.print(matrixLabels[test]+" -> ");    // display label
    float res=neuron.compute(matrixImages[test]);    // compute output
    if (res<0.001) res=0;    // avoid exponential number display
    System.out.println(res);    // display result

}
```

Artificial Intelligence for Robotics: the formal neuron

- **Evaluation**

- We define an error as follow
 - Label is the right number but prediction is <0.5 (false negative)
 - Label is not the right number but prediction is >0.5 (false positive)
- At the end, we display the number of errors (for 10 000 tests)

```
//////////////////////////////////////////
// evaluation of the neuron
int errors=0;

// load evaluation dataset
setData("t10k-images.idx3-ubyte", "t10k-labels.idx1-ubyte");

// TODO : evaluation
for (test=0; test<matrixImages.length; test++){

    System.out.print(matrixLabels[test]+" -> ");    // display label
    float res=neuron.compute(matrixImages[test]);    // compute output
    if (res<0.001) res=0;    // avoid exponential number display
    System.out.println(res);    // display result

    // count errors
    if ( (matrixLabels[test]==number && res<0.5) || (matrixLabels[test]!=number && res>0.5) ) errors++;
}

// display number of errors
System.out.println("total "+errors+" errors");

display.repaint();
```

Artificial Intelligence for Robotics: the formal neuron

- **Evaluation**

- You can try to increase the number of epochs
 - The error decreases, but it seems to be stagnating: increasing the number of epoch will never leads to an error of 0
 - Sometimes, the number of errors increases: the neuron becomes too specialized with training dataset and cannot generalize
 - Overfitting !
- **Your neuron is working !**